

# Project 2: Cryptography

- Due Feb 4 by 11:59pm
- Points 75
- Submitting an external tool
- Available after Jan 26 at 12am

## Part 1: Fun With Keys

### Phase 1 — GPG Public Key (10 pts)

#### 1. Install GPG

- On Ubuntu: `sudo apt install gnupg`
- On macOS (via Homebrew): `brew install gnupg`

#### 2. Generate a Keypair

```
gpg --full-generate-key
```

```
# Choose: (1) RSA and RSA
# Keysize: 4096 bits
# Expiration: (3 months is suggested)
# Enter your full name and Northeastern email when prompted
```

#### 3. Verify Your Key Locally

```
gpg --list-keys
```

**Take Screenshot 1 showing the output of this command.**

#### 4. Upload Your Key to the Key Server

4.1. Run `gpg --list-keys` again and copy your keyID

4.2. Run `gpg --send-keys --keyserver keys.openpgp.org [keyID]`

4.3. Follow the verification email from [keyserver@keys.openpgp.org](mailto:keyserver@keys.openpgp.org) (<mailto:keyserver@keys.openpgp.org>) to publish your key with identity information (your Northeastern email)

4.4. Navigate to <https://keys.openpgp.org/>  (<https://keys.openpgp.org/>)

#### 4.5. Search for your key by entering your Northeastern email

**Take Screenshot 2 showing this page.**

## Phase 2 — Encryption/Decryption (15 pts)

### 1. Construct a message

Create `message.txt` containing a short message of your choice

**Take Screenshot 3 showing the contents of message.txt.**

### 2. Encrypt the message

Teach yourself how to encrypt by reading the `gpg` man page. You should encrypt `message.txt` and store the output in `encrypted_message.gpg`. Make sure `encrypted_message.gpg` can be decrypted using your private key.

**Take Screenshot 4 showing the command you used to encrypt message.txt AND contents of encrypted\_message.gpg.**

### 3. Decrypt the message

Teach yourself how to decrypt by reading the `gpg` man page. You should decrypt `encrypted_message.gpg` and store the output in `decrypted_message.txt`.

**Take Screenshot 5 showing command you used to decrypt encrypted\_message.gpg AND the contents of decrypted\_message.txt.**

## Phase 3 — SSH Key (5 pts)

### 1. Generate an SSH Keypair

```
ssh-keygen -t ed25519
```

```
# Press Enter to accept the default file location when prompted  
# For optimal security, set a passphrase when prompted
```

### 2. Add the SSH Key to GitHub

2.1. Login to GitHub: <https://github.com/>  (<https://github.com/>)

2.2. Navigate to *Settings* → *SSH and GPG keys* → *New SSH Key*

2.3. Title your key: CY2550 Key

2.4. Copy the contents of your public key (likely saved at `~/.ssh/id_ed25519.pub`) and paste into the key field

2.5. Select *Add SSH key*

### 3. Verify Successful Authentication

```
ssh -T git@github.com
```

The expected output is: `Hi [your_username]! You've successfully authenticated, but GitHub does not provide shell access.`

### 4. Authenticate Your Repository

4.1. Navigate to your cy2550 repository, if you are not already there

4.2. Change the remote URL to SSH: `git remote set-url origin git@github.com:[your_username]/cy2550.git`

4.3. Verify the change by running `git remote -v` and ensure it shows: `git@github.com:[your_username]/cy2550.git`

4.4. Test the connection by running `git pull origin main`

**Take Screenshot 6 showing the output of this command.**

## Part 2: Fun With Decoding

### Overview

#### Required Tools

- CyberChef – <https://gchq.github.io/CyberChef/>  (<https://gchq.github.io/CyberChef/>)
- dCode – <https://www.dcode.fr/en>  (<https://www.dcode.fr/en>)

Open and try out these tools before attempting the puzzles. We will not be providing step-by-step tool instruction, be curious and learn the tools on your own.

#### Goal

Decode two multi-layer cipher-texts. Use CyberChef for Puzzle 1 and dCode for Puzzle 2. You will be given the cipher-text and expected to use four schemes per puzzle to decode the message into its plaintext.

### Puzzle 1 (20 pts)

#### Ciphertext

```
GZKoGZKoGZKnGZHuGZHoGZHnGZKuGZHuGHKnGZKuGZKuGHKnGZKuGHHuGZHnGZHuGZHoGZHnGZKuGZHoGZKnGZHuGZHoGZHnG
```

1. Notice all those *repeating characters*? Yeah, they're important. You'll need a key to get started though— try dirt.
2. Looks like there's still more to do. I wish I could just *substitute* this for the answer. Honestly I'm (QWERTY) *bored*.
3. Ugh! That *trailing equals sign* is still there. How can I get rid of it?!
4. Come on. You know what to do with all those *0's and 1s*.

## Ciphertext

## Hints

1. Wait! I think you dropped [this](#)   
(<https://github.com/cy2550repo/cy2550/blob/a9d79b0957552e7634d3bd31b095129e50a350ea/Crypto.jpg>)  
. You'll probably (definitely) need it! Take it as a *substitute* for a hint.
2. Gosh I bet you feel like you're stuck in a *5" wide box* doing these! Make sure to *keep some space*.
3. That was... anticlimactic? Probably a good time to *phone* a friend.
4. I bet you're ready to *(at)bash* your head against your keyboard at this point... hint hint... get it?  
(Don't actually hurt yourself, please!) In fact, you should *reverse* that attitude.

## Extra Credit (+20pts)

You know what to do. Game on.

## Ciphertext

[illegible]

- <https://northeastern.instructure.com/courses/240198/assignments/3110144> 5/6

- **Submit** everything to the Gradescope assignment called *Project 2: Cryptography*. Do not just save your answers— **you must click *Submit & View Submission***.
- **Wait! You're not done just yet...**
  - Verify that your screenshots uploaded correctly and contain the required information. **Fix them now if they do not.**
  - Verify that your two plaintext answers were accepted and are being marked as correct. **Fix them now if they are not.**